

COMPSCI 701:
Creating Maintainable Software
Lecture 2.2: Alterability

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Agenda

- Previously
- Alterability
- Change Cases
- Class Exercise
- Intuition
- Key Points

- Admin
 - Assignment 1
 - `pmd.xml` from Assignment Description on Canvas
 - Cannot find Roster ID — Contact Ewan
 - Schedule — “normal” lecture tomorrow (not discussion only)
 - Discussion posts — one per team, include UPIs
- Alterability—what is it and how do we know we’ve got it
- Lecture preparation
 - Introduction of Software Testing recording (Canvas)

Previously in COMPSCI701

- Agenda
- **Previously**
- Alterability
- Change Cases
- Class Exercise
- Intuition
- Key Points

- The degree of effectiveness and efficiency with which maintenance activities can be carried out on the product. It includes these **independent** sub-attributes:
 - **Comprehensibility** (Analysability)—Degree of effectiveness and efficiency with which the implementation can be understood in order to conduct maintenance activities with confidence
 - **Alterability** (Modifiability)—degree to which a product or system can be effectively and efficiently changed without introducing defects or degrading existing product quality.
 - Testability — degree of effectiveness and efficiency with which test criteria can be established for a system, product or component and tests can be performed to determine whether those criteria have been met.

- Agenda
- Previously
- **Alterability**
- Change Cases
- Class Exercise
- Intuition
- Key Points

Alterability vs Comprehensibility

```

134  rowChoice = io.readInteger("Player 0 choose row: ", 0, 2, -1, "q");
      if (rowChoice < 0) {
          break;
      }
      colChoice = io.readInteger("Player 0 choose col: ", 0, 2, -1, "q");
      if (colChoice < 0) {
          break;
      }
      result = false;
      if (board[rowChoice][colChoice] != ' ') {
          io.println("Invalid move - (" + rowChoice + "," + colChoice +
              ") has already been played");
      } else {
          board[rowChoice][colChoice] = '0';
          result = true;
      }
      while (!result) {
          rowChoice = io.readInteger("Player 0 choose row: ", 0, 2, -1, "q");
          if (rowChoice < 0) {
              break turn;
          }
          colChoice = io.readInteger("Player 0 choose col: ", 0, 2, -1, "q");
          if (colChoice < 0) {
              break turn;
          }
          result = false;
          if (board[rowChoice][colChoice] != ' ') {
              io.println("Invalid move - (" + rowChoice + "," + colChoice +
                  ") has already been played");
          } else {
              board[rowChoice][colChoice] = '0';
              result = true;
          }
      }
167  }

```

- Agenda
- Previously
- **Alterability**
- Change Cases
- Class Exercise
- Intuition
- Key Points

Alterability vs Comprehensibility

```
public class Mark {
    public static final Mark BLANK = new Mark(" ");
    public static final Mark NOUGHT = new Mark("O");
    public static final Mark CROSS = new Mark("X");
    private String _presentation;
    public Mark(String presentation) {
        _presentation = presentation;
    }
    public String present() {
        return _presentation;
    }
    public void render(ASCIIDisplay display, int row, int col) {
        display.render(row, col, _presentation);
    }
    public BoundingBox getBoundingBox() {
        return new BoundingBox(_presentation.length(),1);
    }
    public boolean equals(Object obj) {
        if (obj == null) {
            return false;
        }
        Mark other = (Mark)obj;
        return _presentation.equals(other._presentation);
    }
    public static BoundingBox maxBoundingBox() {
        BoundingBox result = Mark.BLANK.getBoundingBox();
        result = result.union(Mark.NOUGHT.getBoundingBox());
        result = result.union(Mark.CROSS.getBoundingBox());
        return result;
    }
}
```

Alterability

- Agenda
- Previously
- **Alterability**
- Change Cases
- Class Exercise
- Intuition
- Key Points

- **Alterability**—degree to which a product or system can be effectively and efficiently changed without introducing defects or degrading existing product quality. (Lecture 1.1)
 - “effectively and efficiently” **not** counting costs associated with comprehension
 - it can be that some code is “hard” to understand but “easy” to change
 - so when assessing alterability assume the mental model is “complete” (as much as this makes sense to do so)

- Agenda
- Previously
- **Alterability**
- Change Cases
- Class Exercise
- Intuition
- Key Points

Software quality attributes like Alterability

ISO/IEC 9216 Maintainability has subcharacteristics Analyzability, **Changeability**, Stability, Testability.

Changeability—Characterizes the amount of effort to change a system.

ISO/IEC 25010 Maintainability has subcharacteristics Modularity, Reusability, Analyzability, **Modifiability**, Testability.

Modifiability—Degree to which a product or system can be effectively and efficiently modified without introducing defects or degrading existing product quality.

IEEE Standard Glossary of Software Engineering Terminology , 1990 Maintainability, Extendability, Flexibility

Maintainability The ease with which a software system or component can be **modified** to correct faults, improve performance or other attributes, or adapt to a changed environment.

Extendability The ease with which a system or component can be **modified** to increase its storage or functional capacity.

Flexibility The ease with which a system or component can be **modified** for use in applications or environments other than those for which it was specifically designed.

Bass, Clements, Kazman a system is typically **modifiable** if changes involve the fewest possible number of distinct elements (Software Architecture in Practice)

What is involved in making a change to code?

- Agenda
- Previously
- **Alterability**
- Change Cases
- Class Exercise
- Intuition
- Key Points

1. Find *where* the changes need to be made (Comprehension)
2. Determine *what* changes need to be made (Comprehension)
3. Make the changes (Alteration)

Change Cases: Specifying what needs to change

- Agenda
- Previously
- Alterability
- Change Cases
- Class Exercise
- Intuition
- Key Points

- From: Ecklund, Delcambre, Freiling *Change Cases: Use Cases that Identify Future Requirements* OOPSLA'96.
- describe new potential *requirements* for a system or modifications to existing *requirements*
- include **likelihood** the case will occur, and what the potential **impact** is likely to be
 - Impact is assessed against the **requirements**, not the existing code
 - Impact is determined by how much the **behaviour of the existing system** will be changed by the change case
 - Heuristic: **What proportion of the original “cost” of the product are you prepared to pay for this change case?**
(cost may be measured in financial terms, or time, or effort, or in some other way)
- The **lower the likelihood** of a change case being required, then less affect it will have on the alterability assessment
- The **lower the impact** of a change case being required, then the less affect it will have on the alterability assessment
- **⇒ focus on change cases with high likelihood and high impact**

Class Exercise

- Agenda
- Previously
- Alterability
- Change Cases
- **Class Exercise**
- Intuition
- Key Points

1. Form teams.
2. Goal: Compare the alterability of NoughtsAndCrosses 2.1 and 1.3
3. Which implementation do you think has better alterability?
4. Find a change case with high likelihood and high impact
5. Determine which implementation will have lower cost in implementing the change case
6. Find a change case that implementation 2.1 has better alterability than 1.3. How easy was it to find one?
7. Find a change case that implementation 1.3 has better alterability than 2.1. How easy was it to find one?
8. Post to the relevant discussion on Canvas

- Agenda
- Previously
- Alterability
- Change Cases
- Class Exercise
- **Intuition**
- Key Points

Intuition for Alterability

- Unlike comprehension there hasn't been much done to create a “model of alterability”, so what's our intuition?
 - For a given change case, all things being equal‡ . . .
 - It is likely to be less work to make all the required changes in one class, compared with having to make changes in multiple classes.
 - because as well as the time taken to change the code, there is also the time taken to switch between classes, and switching between classes increases the chances of making a mistake—remember “without introducing defects or degrading existing product quality”
 - It is likely to be less work to make all the required changes in one method in a class, compared with having to make changes in multiple different methods in the same class
 - because, switching again
 - It is likely to be less work to only have to change one statement in a method, compared with having to change multiple statements
 - because, switching again
 - It is likely to be less work if the change that needs to be made is “simple,” compared with having to make a “complex” change
 - e.g. add a new branch to a switch statement that just does a simple variation on what all the other branches are doing, compared with changing the expression in the condition of a loop— remember “without introducing defects or degrading existing product quality”
- (‡ which of course they're not)

Intuition for Alterability (cont)

- Agenda
- Previously
- Alterability
- Change Cases
- Class Exercise
- **Intuition**
- Key Points

- Generally, the less we have to change **existing code**, the more alterable the design is:
 - fewer places we need to deal with (switching — **efficiency**)
 - less likely to make a mistake (**effectiveness**)
- What about one simple change in each of 3 classes compared with 3 complex changes to statements?

- Agenda
- Previously
- Alterability
- Change Cases
- Class Exercise
- Intuition
- Key Points

Key Points

- Alterability is a quality attribute for the design of the software
- Change cases provides a way to talk about what kinds of changes may be needed, and so assess the alterability of the design
- Generally want to minimise the “amount” of existing code that needs to be changed.